



University Of Frontier Technology,Bangaldesh

Department of Cyber Security Engineering

PROJECT REPORT

Project Name:SafeVault: An Object-Oriented Secure Finance Framework

Course Name:Object Oriented Programming Sessional

Course Code: PROG 112

Submitted By: 1.Ariful Islam(2404033) 2.Anika Akter Nira(2404034) Level:1 Term:2 Department of CYSE,UFTB	Submitted To: Md.Masud Rana Lecturer Department of CYSE, UFTB
--	--

SafeVault: An Object-Oriented Secure Finance Framework

1.Introduction: SafeVault is a simple finance management system developed using C++ and Object-Oriented Programming (OOP). The main purpose of this project is to help users manage their daily income, expenses, budgets, and reminders in a secure and organized way. Nowadays many students and individuals cannot properly track their spending. Because of this, they often face financial problems. SafeVault helps users record income, expenses, bills, and reminders so they can manage money more efficiently. The project also includes a login system for security and file handling for permanent data storage.

2.Problem Statement: Many people forget where they spend money and cannot maintain a proper budget. Traditional methods like writing expenses on paper are difficult to manage and insecure. Students and hostel residents especially need a simple system to manage their daily financial activities. This project solves the problem by providing a user-friendly and secure finance management system.

3.Proposed Solution: SafeVault provides a simple solution for personal finance management. The system allows users to:

- Securely log into the application
- Record daily income and expenses
- Monitor current balance automatically
- Set monthly budget limits
- Receive warnings when the budget limit is almost reached
- Split bills equally among friends
- Store data permanently using text files
- Export reports for future use

4.Objectives:

1. To develop a secure personal finance management system.
2. To implement Object-Oriented Programming concepts practically.
3. To create a simple and user-friendly interface.
4. To provide authentication using username and password.
5. To track income, expenses, and savings.
6. To store user data permanently using file handling.

7. To generate financial reports for users.
8. To improve budgeting awareness among students and individuals.

5. System Requirements & Architecture:

- 1) **Program Language:** C++
- 2) **Operating System:** Windows 11
- 3) **IDE/Editor:** VS code/Code::Blocks

Flowchart: Start → Login → Main Menu → Select Feature → Process → Save Data → Output

6. OOPS Concept Used:

- ✓ **Classes and Objects:** The project uses classes and objects to organize the program properly.
- ✓ **Encapsulation:** Private variables are used to protect sensitive data such as financial information.
- ✓ **Inheritance:** The SafeVault class inherits username and password information from the User class.
- ✓ **Constructor:** A constructor is used to automatically load previous data and display a welcome message when the object is created.
- ✓ **Abstraction:** Users only interact with menu options while internal calculations remain hidden.
- ✓ **Modular Programming:** Different tasks are divided into separate functions such as: login(), addIncome(), addExpense(), setBudget(), reminder(), splitBill()

7. Program Code:

```
#include<iostream>

#include<fstream>

using namespace std;

class User {

protected:

    string username = "admin";

    string password = "1234";};

class SafeVault : public User {
```

private:

float income = 0;

float expense = 0;

float budget = 0;

public:

SafeVault() {

loadData();

cout << "Welcome to SafeVault System\n";

}

void loadData() {

ifstream file("report.txt");

if(file.is_open()) {

file >> income;

file >> expense;

file >> budget;

file.close(); }

}

void saveData() {

ofstream file("report.txt");

file << income << endl;

file << expense << endl;

file << budget << endl;

file.close();

}

bool login() {

```

string user, pass;

int attempt = 0;

while(attempt < 3) {

    cout << "Username: ";

    cin >> user;

    cout << "Password: ";

    cin >> pass;

    if(user == username && pass == password) {

        cout << "\nLogin Successful!\n";

        return true; }

    else { cout << "\nWrong Password!\n";

        attempt++; }

} cout << "\nAccess Denied!\n";

return false;

}

void addIncome() {

    float amount;

    cout << "Enter Income: ";

    cin >> amount;

income += amount;

    saveData();

    cout << "Income Added Successfully!\n";

}

void addExpense() {

    float amount;

```

```

    cout << "Enter Expense: ";

    cin >> amount;

    expense += amount;

    saveData();

    cout << "Expense Added Successfully!\n";

}

void showBalance() {

    float balance = income - expense;

    cout << "\n===== FINANCIAL SUMMARY =====\n";

    cout << "Total Income : " << income << " Taka\n";

    cout << "Total Expense : " << expense << " Taka\n";

    cout << "Current Balance : " << balance << " Taka\n";

    if(expense >= budget && budget != 0) {

        cout << "\nWarning! Budget Limit Reached!\n"; }

    }

void setBudget() {

    cout << "Enter Budget Limit: ";

    cin >> budget;

    saveData();

    cout << "Budget Saved Successfully!\n";

}

void splitBill() {

    float bill;

    int people;

    cout << "Enter Total Bill: ";

```

```

    cin >> bill;

    cout << "Enter Number of People: ";

    cin >> people;

    cout << "\nEach Person Will Pay: "<< bill / people << " Taka\n";

}

void reminder() {

    string name;

    string date;

    float amount;

    cin.ignore();

    cout << "\nEnter Bill or Debt Name: ";

    getline(cin, name);

    cout << "Enter Amount: ";

    cin >> amount;

    cout << "Enter Due Date: ";

    cin >> date;

    cout << "\n===== REMINDER SAVED =====\n";

    cout << "Pay " << amount << " Taka for " << name << " before " << date
<< endl;

}

void exportReport() {

    ofstream file("finance_report.txt");

    file << "===== SafeVault Report =====\n";

    file << "Income : " << income << endl;

    file << "Expense : " << expense << endl;

    file << "Balance : " << income - expense << endl;

```

```

file.close();

    cout << "\nReport Exported Successfully!\n"; }

};

int main() {

    SafeVault obj;

    if(obj.login() == false) {

        return 0; }

    int choice;

    do {

        cout << "\n===== SafeVault MENU =====\n";

        cout << "1. Add Income\n";

        cout << "2. Add Expense\n";

        cout << "3. Show Balance\n";

        cout << "4. Set Budget\n";

        cout << "5. Split Bill\n";

        cout << "6. Add Reminder\n";

        cout << "7. Export Report\n";

        cout << "8. Exit\n";

        cout << "Enter Choice: ";

        cin >> choice;

        switch(choice) {

            case 1:

                obj.addIncome();

                break;

            case 2:

```

```

    obj.addExpense();

    break;

case 3:

    obj.showBalance();

    break;

case 4:

    obj.setBudget();

    break;

case 5:

    obj.splitBill();

    break;

case 6:

    obj.reminder();

    break;

case 7:

    obj.exportReport();

    break;

case 8:

    cout << "\nProgram Ended Successfully!\n";

    break;

default:

    cout << "\nInvalid Choice!\n";}

} while(choice != 8);

return 0;}

```

8.Sample Output

i. Login Output:

(if enter wrong password or username 3 times)

```
"C:\Users\Anika Akter Nira" x + v
== SAFEVAULT LOGIN ==
Enter Username: admin
Enter Password: password
Wrong details! Attempts remaining: 2

== SAFEVAULT LOGIN ==
Enter Username: admin
Enter Password: Password
Wrong details! Attempts remaining: 1

== SAFEVAULT LOGIN ==
Enter Username: Admin
Enter Password: Admin
Wrong details! Attempts remaining: 0

Security Alert: 3 wrong attempts reached. Program Closing!
```

(if enter right password and username)

```
"C:\Users\Anika Akter Nira" x + v
=====
WELCOME TO SAFEVAULT SYSTEM
=====
Enter Username: admin
Enter Password: 1234

>>> [SUCCESS] Login Verified!

----- SYSTEM ALERT LOGS -----
* No active loan repayments or bill deadlines found.
-----
```

ii. Income Output:

```
===== SafeVault MENU =====
1. Add Income
2. Add Expense
3. Show Balance
4. Set Budget
5. Split Bill
6. Add Reminder
7. Export Report
8. Exit
Enter Choice: 1
Enter Income: 10000
Income Added Successfully!
```

iii. Expense Output:

```
===== SafeVault MENU =====
1. Add Income
2. Add Expense
3. Show Balance
4. Set Budget
5. Split Bill
6. Add Reminder
7. Export Report
8. Exit
Enter Choice: 2
Enter Expense: 200
Expense Added Successfully!
```

iv. Show Balance Output:

```
===== SafeVault MENU =====
1. Add Income
2. Add Expense
3. Show Balance
4. Set Budget
5. Split Bill
6. Add Reminder
7. Export Report
8. Exit
Enter Choice: 3
```

```
===== FINANCIAL SUMMARY =====
Total Income : 10000 Taka
Total Expense : 200 Taka
Current Balance : 9800 Taka
```

v. Set Budget Output:

```
===== SafeVault MENU =====
1. Add Income
2. Add Expense
3. Show Balance
4. Set Budget
5. Split Bill
6. Add Reminder
7. Export Report
8. Exit
Enter Choice: 4
Enter Budget Limit: 10000
Budget Saved Successfully!
```

(Budget Warning!)

```
===== SafeVault MENU =====
1. Add Income
2. Add Expense
3. Show Balance
4. Set Budget
5. Split Bill
6. Add Reminder
7. Export Report
8. Exit
Enter Choice: 3

===== FINANCIAL SUMMARY =====
Total Income : 10000 Taka
Total Expense : 9900 Taka
Current Balance : 100 Taka

Warning! Budget Limit Reached!
```

vi. Split bill Output:

```
===== SafeVault MENU =====
1. Add Income
2. Add Expense
3. Show Balance
4. Set Budget
5. Split Bill
6. Add Reminder
7. Export Report
8. Exit
Enter Choice: 5
Enter Total Bill: 1600
Enter Number of People: 5

Each Person Will Pay: 320 Taka
```

vii. Reminder Output:

```
===== SafeVault MENU =====
1. Add Income
2. Add Expense
3. Show Balance
4. Set Budget
5. Split Bill
6. Add Reminder
7. Export Report
8. Exit
Enter Choice: 6

Enter Bill or Debt Name: internet
Enter Amount: 100
Enter Due Date: 24/05/26

===== REMINDER SAVED =====
Pay 100 Taka for internet before 24/05/26
```

viii. Export Report:

```
===== SafeVault MENU =====
1. Add Income
2. Add Expense
3. Show Balance
4. Set Budget
5. Split Bill
6. Add Reminder
7. Export Report
8. Exit
Enter Choice: 7

Report Exported Successfully!
```

ix. Exit:

```
===== SafeVault MENU =====
1. Add Income
2. Add Expense
3. Show Balance
4. Set Budget
5. Split Bill
6. Add Reminder
7. Export Report
8. Exit
Enter Choice: 8

Program Ended Successfully!
```

9. Advantages:

- ✓ Easy and simple to use
- ✓ Beginner friendly system
- ✓ Demonstrates important OOP concepts clearly
- ✓ Reduces the risk of forgetting payments
- ✓ Organized menu-driven system
- ✓ Useful for students and hostel residents
- ✓ Improves personal financial management skills
- ✓ Lightweight and fast program execution

10. Limitations:

- i. The project uses a command line interface only
- ii. No graphical user interface (GUI) is available
- iii. Supports only a single user and no database integration system
- iv. Financial data is stored in simple text files only
- v. No advanced data encryption for security

vi. Large-scale financial management is not supported

11.Future Scope:

1. Graphical User Interface (GUI)
2. Database integration system
3. Mobile application version
4. Multi-user login support
5. Real-time notification and reminder system
6. Advanced password encryption
7. Online payment and transaction system

12.Conclusion: SafeVault is a simple finance management system developed using C++ and OOP concepts. The project helps users manage income, expenses, budgets, and reminders easily. It also stores data permanently using file handling. Overall, this project is useful for learning practical applications of Object-Oriented Programming.